Appendix B: Raw decoder excerpt *preprocessing/pingstation_raw_decoder.py*

```
import json
import re
import socket
import struct
import threading
import time
from datetime import datetime, timedelta
from pathlib import Path


def _now_iso():
    return datetime.utcnow().isoformat() + "Z"


class PingstationRawCaptureLogger:
    """
    Logs each session at three processing stages:
    - <date_time>_engi_H.txt (hex lines)
    - <date_time>_engi_A.txt (ascii lines)
    - <date_time>_engi_RAW.txt (decoded records before preprocessing)
    """

    def __init__(self, file_path):
        target = Path(file_path)
        self.capture_dir = target if target.suffix == "" else target.parent
        self.capture_dir.mkdir(parents=True, exist_ok=True)
        self.hex_dir = self.capture_dir / "hex_data"
        self.ascii_dir = self.capture_dir / "ascii_data"
        self.hex_dir.mkdir(parents=True, exist_ok=True)
        self.ascii_dir.mkdir(parents=True, exist_ok=True)
        stamp = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
        self.hex_file_path = self.hex_dir / f"{stamp}_engi_H.txt"
        self.ascii_file_path = self.ascii_dir / f"{stamp}_engi_A.txt"
        self.raw_file_path = self.capture_dir / f"{stamp}_engi_RAW.txt"
        self.stream_file_path = self.capture_dir / f"{stamp}_engi_STREAM.bin"
        self.hex_file_path.touch(exist_ok=True)
        self.ascii_file_path.touch(exist_ok=True)
        self.raw_file_path.touch(exist_ok=True)
        self.stream_file_path.touch(exist_ok=True)
        self._lock = threading.Lock()

    def _extract_ascii_tokens(self, payload):
        text = payload.decode("ascii", errors="ignore")
        return re.findall(r"[A-Za-z0-9]{3,}", text)

    def log_chunk(self, payload, source_ip, source_port):
        if not payload:
            return

        hex_data = payload.hex()
        ascii_tokens = self._extract_ascii_tokens(payload)
        ascii_line = ", ".join(ascii_tokens) if ascii_tokens else "None"

        with self._lock:
            with self.hex_file_path.open("a", encoding="utf-8") as hf:
                hf.write(hex_data + "\n")
            with self.ascii_file_path.open("a", encoding="utf-8") as af:
                af.write(ascii_line + "\n")
            # Preserve the exact receiver stream separately for protocol debugging.
            with self.stream_file_path.open("ab") as rf:
                rf.write(payload)

    def log_decoded(self, record):
        """Append one decoded record before preprocessing as JSON Lines text."""
        if not isinstance(record, dict) or not record:
            return
        clean_record = {
            key: value
            for key, value in record.items()
            if not str(key).startswith("_")
        }
        with self._lock:
            with self.raw_file_path.open("a", encoding="utf-8") as rf:
                rf.write(json.dumps(clean_record, ensure_ascii=True, separators=(",", ":")) + "\n")
```